

Formalizing Mathematics at Scale

Ahmad Rammal^{1,2}, Niket Patel^{1,3}, Fabian Gloeckle^{1,†,2}, Amaury Hayat^{2,4}, Julia Kempe^{1,3}, Remi Munos¹, Charles Arnal^{1,*}, Vivien Cabannes^{1,*}

¹FAIR at Meta, ²CERMICS, ENPC, Institut Polytechnique de Paris, ³New York University, ⁴Korea Institute for Advanced Study

*Equal contribution, [†]Work done while at Meta

We present AUTOFORMBOT, a multi-agent system for building an **Autoformalized Textbook Library At Scale (ATLAS)** in Lean 4. AUTOFORMBOT orchestrates thousands of LLM agents, equipped with formal verification tools, dependency-aware task scheduling, and collaborative version control, to translate informal textbook prose into machine-checked definitions and proofs. We apply our methods to a corpus of **26** open-access textbooks spanning analysis, algebra, topology, combinatorics, and probability, producing ATLAS: a verified library of over **45,000** Lean 4 declarations and **500 thousand** lines of code. We release two artifacts: (i) AUTOFORMBOT, the open-source multi-agent framework; and (ii) ATLAS, the resulting formal library. Our results suggest that autoformalizing the core content of graduate-level mathematics at scale is now economically and technically feasible. This opens the door to the automated verification of both human- and machine-generated mathematics at a research level.

Date: May 28, 2026

Correspondence: Ahmad Rammal at rammal@meta.com

Code:

<https://github.com/facebookresearch/autoform-bot>

<https://github.com/facebookresearch/atlas-lean>



1 Introduction

As large language models accelerate the production of ideas, code, and other artifacts, the *verification* of their output becomes a major bottleneck. In mathematics, this bottleneck is particularly acute: the peer review system is already under pressure, and referees increasingly lack the time to check every step of a proof, instead relying on their mathematical intuition to judge whether the proof *appears* credible. Once AI systems start generating mathematical reasoning at a pace that dwarfs human capacity, this trust-based verification model will become untenable. The recent *First Proof challenge* (Abouzaid et al., 2026; Armstrong et al., 2026) provided a glimpse of this future: LLMs rapidly generated dozens of solutions to challenging research questions, a few of which were correct and most of which were wrong in subtle, nontrivial ways. Proof assistants offer a way out.

Proof assistants In a proof assistant, users express mathematical definitions, theorem statements, and proofs in a specialized programming language with a formal foundation, a process called *formalization*; a small engine (the *kernel*) checks every logical step mechanically, so that any accepted proof is guaranteed to be valid, relative to the stated definitions and assumptions. Thus, verifying a formalized proof reduces to checking that it is accepted by the proof assistant and that the formal statement itself faithfully captures the intended meaning. A simple example of a formalized statement is given in Figure 1. We use Lean 4 (de Moura and Ullrich, 2021), a modern proof assistant that doubles as a general-purpose programming language, as our formalization environment. Other widely used proof assistants include, for instance, Rocq/Coq (Bertot and Castéran, 2004), Isabelle/HOL (Nipkow et al., 2002), HOL Light (Harrison, 2009), Agda (Bove et al., 2009), Mizar (Grabowski et al., 2010; Bancerek et al., 2018), and Metamath (Megill and Wheeler, 2019).

For all $x \in \mathbb{N}$, if $x < 2$, then $x + 3 < 5$.	<code>example (x : Nat) (h : x < 2) : x + 3 < 5 :=</code>
Let $x < 2$. Adding 3 to both sides preserves the inequality, so $x + 3 < 2 + 3 = 5$.	<code>by</code> <code> exact Nat.add_lt_add_right h 3</code>

Figure 1 A simple mathematical statement and proof, shown informally and as a Lean formalization.

Laying out foundations Formalizing mathematics is cumulative: before one can state a theorem about, say, compact metric spaces, one needs formal definitions of metrics, topological spaces, compactness, and the lemmas connecting them. MATHLIB (The Mathlib Community, 2020), Lean’s community-maintained mathematical library, is an ongoing effort to build such foundations. With roughly 2.1 million lines of code and years of expert effort behind it, MATHLIB covers broad but incomplete swaths of the mathematical landscape; while some domains, such as algebra or category theory, are well-represented, large gaps remain in others, such as differential geometry or PDEs. As a result, most current mathematical research cannot be formalized in Lean without prohibitively time-consuming preparatory work.

Textbooks are a natural unit of work for closing this gap: by their very nature, they provide solid and general foundations upon which to build. While formalizing an entire textbook is a tremendous undertaking for a human expert, recent progress in the capabilities of frontier models has opened the door to the automated formalization, or *autoformalization*, of large corpora of texts (Wu et al., 2022; Urban, 2026; Wang et al., 2026; Gloeckle et al., 2026).

Benefits of autoformalization at scale Efficient automated formalization would make large-scale LLM-powered mathematical research possible: large groups of human experts and LLM agents could collaborate in a process reminiscent of open-source software, working independently on modular, well-specified pieces while the proof assistant ensures that their contributions compose correctly. Without such mechanical verification, the resulting body of work would exceed any individual’s capacity to check. It would additionally allow for the validation of existing mathematical knowledge, which has been one of the main goals of formalization efforts so far. Finally, autoformalization could be used to produce trustworthy reward signals for mathematical reasoning trajectories when training LLMs with reinforcement learning. Indeed, current methods, which involve training on problems with explicit and easily verifiable solutions (“2”, “ $\sqrt{x+1}$ ”, ...) or relying on other LLMs as judges (Cobbe et al., 2021; Hendrycks et al., 2021; Zheng et al., 2023), might not scale to the research-level mathematics that frontier models are starting to tackle (Feng et al., 2026).

Multi-agent scaffolds While frontier models have made considerable progress in Lean in recent years, they still trail behind in performance compared to other programming languages; Claude Opus 4.6, Gemini 3.1 Pro, or GPT 5.4 (Anthropic, 2026; Google DeepMind, 2026; OpenAI, 2026) typically fail in their initial attempt to provide a 20-line proof to a standard mathematical statement, but eventually succeed when given enough time and tools. As such, the formalization of an average textbook is a codebase-building endeavor far beyond what LLMs can achieve in a single, unsupervised shot. This creates the need for adapted scaffolds that provide tools, frameworks, and supervision and can leverage parallelism by coordinating the work of dozens or hundreds of agents.

Contributions In this work, we introduce AUTOFORMBOT, a multi-agent framework for formalizing mathematical textbooks at scale, whose agents are powered through user-supplied access to frontier-model APIs. Scaling from a single coding agent to thousands working on a shared repository raises hard coordination problems: agents make incompatible design decisions, generate duplicate work, pursue tangential goals, and create cascading failures through a shared merge queue. AUTOFORMBOT treats formalization as a collaborative software engineering problem, coordinating agents through mechanisms that are well represented in their training data: git branches, pull request review, and issue tracking. As formal verification provides a sharp coordination signal, textbook formalization becomes an ideal testbed for multi-agent orchestration research.

We release two artifacts:

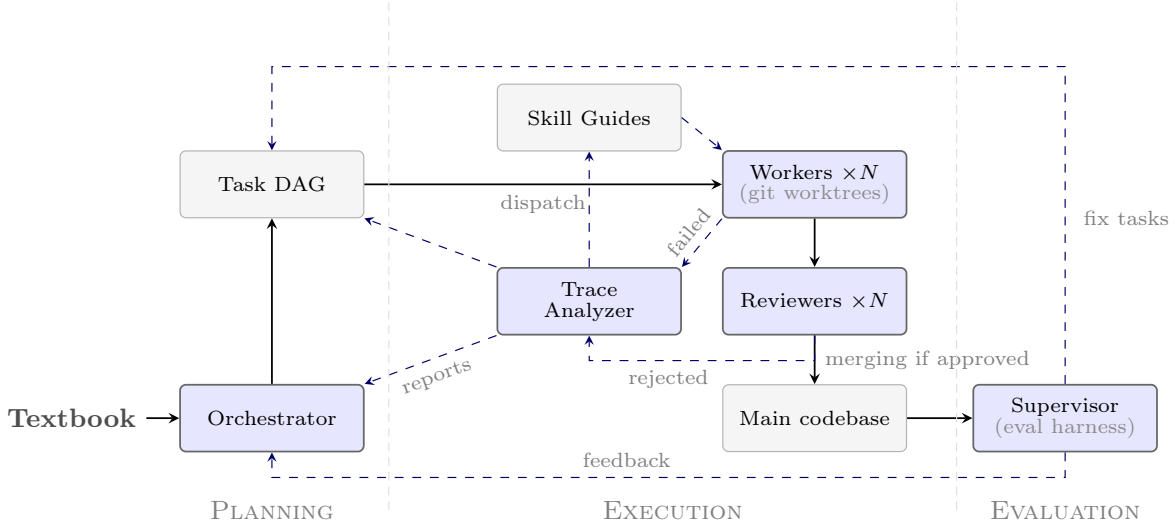


Figure 2 AUTOFORMBOT architecture. The orchestrator reads the book and builds a task DAG. Workers formalize individual statements in isolated worktrees; successful builds pass through concurrent review and a batched merge queue. The trace analyzer learns from task failures and writes skill guides for subsequent attempts. The supervisor evaluates book targets after each merge and dispatches fix tasks via triage agents. Arrows indicate information flow; dashed arrows indicate feedback loops.

1. **AUTOFORMBOT: an open-source multi-agent framework** for textbook autoformalization, featuring formal verification tools, dependency-aware task scheduling, and collaborative version control. It includes a visualizer for human-in-the-loop interaction, a rigorous evaluation pipeline, and can be easily connected to user-supplied models, whether accessed through APIs or hosted locally.
2. **ATLAS: verified formal libraries at scale.** We apply AUTOFORMBOT, powered primarily by Opus 4.6, to **26** open-access mathematical textbooks and produce verified Lean 4 libraries totaling over **45,000** declarations spanning several areas of mathematics and extending MATHLIB’s coverage. We intend to keep improving this initial effort until it becomes complete and systematic enough to serve as a seamless machine-generated extension of MATHLIB.

Beyond the artifacts, we report empirical observations on multi-agent coordination at scale: an adversarial dynamic between workers and reviewers around verification circumvention, context degradation in long-lived agents that motivates delegation to specialized short-lived ones, and ablations quantifying the contribution of each feedback component.

2 Related Work

LLM-based theorem proving and autoformalization. A growing line of work applies language models to formal proof generation. GPT- f (Polu and Sutskever, 2020; Polu et al., 2022) first demonstrated that transformers can generate useful proof steps in formal languages. Lample et al. (2022) further showed that combining transformers with a reinforcement learning approach could strongly increase performance. Subsequent systems such as LEGO-Prover (Wang et al., 2024), AlphaProof (Hubert et al., 2025), ABEL (Gloeckle et al., 2024) or more recently DeepSeek-Prover (Xin et al., 2024; DeepSeek-AI, 2025), Goedel-Prover (Lin et al., 2025), Kimina-Prover (Wang et al., 2025), Seed-Prover (Chen et al., 2025) have been developed and pushed the state of the art on benchmarks like miniF2F (Zheng et al., 2022), ProofNet (Azerbaiyev et al., 2023), the International Mathematical Olympiads, or Putnam Bench (Tsoukalas et al., 2024). Other systems such as Achim et al. (2025) and Chen et al. (2026) have been able to provide formal proofs to research-level questions.

On the autoformalization side, since Wu et al. (2022), a large body of work has studied autoformalization datasets (e.g. Azerbaiyev et al., 2023; Ying et al., 2024; Ju and Dong, 2026; Liu et al., 2026). Among the

milestones, one can cite, for instance, large-scale formalizations of competition problems (Hubert et al., 2025; AI-MO, 2025) or, recent hybrid approaches between manual and automatic formalization leading to the formalization of the strong prime number theorem, the sphere packing theorem (Hariharan et al., 2026) or De Giorgi-Nash-Moser theory (Armstrong and Kempe, 2026). These approaches target isolated theorems; none attempts the systematic formalization of a full textbook.

Large-scale textbook formalization. Several recent efforts have begun to tackle longer documents. Urban (2026) used a single coding agent to formalize initial chapters of a point-set topology textbook, in Megalodon. Wang et al. (2026) presented a two-phase multi-agent scaffold formalizing an introductory analysis textbook of 300 pages producing 40k lines of Lean, though with a significant MATHLIB overlap. Gloeckle et al. (2026) provided the first example of graduate textbook fully autoformalized without human intervention in one week. Our work differs from these works both in scale and methodology: we formalize **26** books across diverse mathematical areas, using an open-source framework with detailed compute cost and efficiency reporting.

Multi-agent systems for software engineering. SWE-agent (Yang et al., 2024) demonstrated effective single-agent software engineering, while MetaGPT (Hong et al., 2024) introduced role-based multi-agent coordination with structured communication.

3 Success Criteria and Evaluation Harness

Before describing the pipeline, we define what constitutes a successful formalization and the harness used to measure it. In Lean, one can avoid providing a proof by declaring it an axiom or by replacing the proof with the `sorry` keyword. Our policy is that every statement whose proof appears in the source material must be properly formalized, without `sorry` or axioms; statements whose proof is not provided (e.g. because the author defers to a reference) may be axiomatized. However, an autoformalization attempt can fail in several ways. It may fail by illegitimately using an axiom or `sorry`. More subtly, a statement may appear fully proved but depend on another declaration whose proof contains an axiom that propagates silently through the dependency chain. Finally, an agent may produce structurally degenerate formalizations that technically compile but do not faithfully capture the underlying mathematics (e.g. a theorem proved only for $\mathbb{F}_p$ instead of for all groups, or a definition that bakes in what should be a separate theorem).

We start each attempt by running a preprocessing script that identifies all formalizable statements in the source textbook; we call these *target statements* or *goals*. A statement is successfully formalized if it faithfully captures the mathematical content and its proof does not directly rely on any illegitimate axiom or `sorry`. This criterion is non-transitive: if a correct proof invokes a target lemma whose own proof contains a `sorry`, the calling statement is deemed successful but the lemma is not. Our main success metric is the number of successfully formalized target statements.

Detecting illegitimate axiomatizations requires recursively checking the declarations a statement depends on, rather than inspecting individual declarations in isolation. A direct `sorry` is easy to spot, but an axiom or weakened definition that propagates through a chain of helper lemmas can only be caught by analyzing the dependency structure of the entire project. To this end, we build a *declaration dependency graph* by running a Lean metaprogram inside the compiled project (Figure 3). The metaprogram walks every project-local declaration and extracts its nature (theorem, axiom, etc.), the set of project-local declarations it references, and its axiom set. For each declaration, we also compute *structural tags* by inspecting the proof term, flagging patterns that may indicate faithfulness defects: vacuous bodies, hypothesis smuggling, trivially constructed instances, and others.

The evaluation harness uses the dependency graph together with a set of mechanical and LLM-based checks, applied in three stages:

1. **Mechanical gates.** The project must compile without errors, and source files must not contain metaprogramming keywords (`elab`, `syntax`) that could make the code semantically misleading.
2. **Matching.** For each target statement, a matcher agent finds its corresponding Lean declaration in the generated codebase.

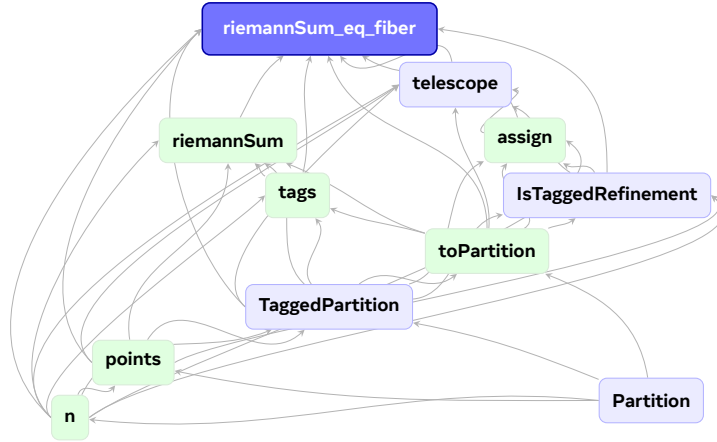


Figure 3 Dependency cone of a target declaration. Edges represent direct dependencies between project-local declarations. Structural tags and gap markers propagate upward as alerts on the target.

3. **Statement-level grading.** Three independent LLM judges score each matched target along the rubrics described below.

The dependency graph is exposed to the judges as a queryable tool, allowing them to trace sorry chains, inspect structural tags, and investigate suspicious dependencies when scoring. The three rubrics are:

1. **Faithfulness:** whether the Lean statement captures the source material’s mathematical content (hypotheses and conclusions) at full strength, without weakening the result or hiding content in typeclass fields.
2. **Proof integrity:** whether the proof chain represents genuine mathematical work—no unjustified `sorry` or axioms, no orphan classes used as hypotheses, no vacuous definitions.
3. **Code quality:** adherence to Mathlib conventions (using intelligence collected by Vilin, 2026)—naming, tactic choice, typeclass generality, and proof structure.

Our evaluation harness is released as a self-contained module in the accompanying codebase.

4 AutoformBot

AUTOFORMBOT casts textbook formalization as a collaborative software engineering problem. From a software engineering perspective, a formalization project is simply a code repository in a special-purpose programming language (Lean 4) that must build without errors and pass quality checks regarding faithfulness to the source material. Coding agents—persistent LLM instances equipped with code-related tools—fit this task well: they can read files, make edits, run terminal commands, and interact with the Lean type checker. The challenge is scaling from one agent to thousands while maintaining coherence. AUTOFORMBOT is an agentic harness meant to address this challenge by equipping individual agents—powered through access to any served model—with tools geared towards formal verification capabilities, a shared infrastructure for task tracking and version control, and a flexible collaboration architecture.

4.1 Pipeline Architecture

The pipeline is a multi-agent system organized in three management tiers (Figure 2): a high-level *orchestrator* that plans work from the book’s mathematical structure, mid-level agents—the *trace analyzer* and the *supervisor*—that manage learning at the task level and evaluation at the target level, and low-level *workers* and *reviewers* that execute individual formalization tasks. These tiers communicate through a shared coordination infrastructure described further below in §4.3. Each agent is powered through access to the endpoint of a served model—typically Opus 4.6 in our experiments. This architecture sets it apart from the earlier, more loosely-organized system in Gloeckle et al. (2026).

Planning: the orchestrator. The orchestrator is a long-lived LLM agent that reads the source textbook and encodes the work to be done as a task Directed Acyclic Graph (DAG), whose nodes are formalization targets (definition, theorem, etc.) and whose edges encode the book’s logical dependencies—if theorem B uses definition A, then the task for B depends on the task for A. It continuously updates the DAG as the project evolves, and maintains a persistent TODO list on disk to track patterns and issues across rounds, compensating for the limited context window of a long-running conversation.

Execution: workers and reviewers. Tasks are dispatched by a runner that continuously polls for ready tasks (all dependencies met) and assigns them to available workers. A worker’s output must pass a series of quality checks (see Section 3), including inspection by a *reviewer*. Optionally, multiple workers can race on the same target, each in its own worktree; the first to clear all quality gates wins and the remaining attempts are cancelled. Approved changes enter a merge queue (see §4.3).

Task-level feedback: the trace analyzer. When a task fails, the system must learn from the failure so that the next attempt does not repeat the same mistakes. The *trace analyzer* is a persistent agent assigned to each failed task. It maintains *skill guides* containing lessons from past attempts; workers are required to read this guide before starting a new attempt. It acts on the DAG based on feedback from the workers about task difficulty and potential decompositions, and generates a structured report summarizing the failure, which the orchestrator consumes during its next planning round.

Target-level feedback: the supervisor. The *supervisor* loop operates at the level of the list of targets, driven by a *goal tracker* that records the status of each target statement (pending, completed, or failed) independently of the task DAG. After each successful merge, the supervisor computes the git diff, uses a matcher agent to identify which target statements were affected, and runs the evaluation harness (Section 3) on the affected targets in an isolated worktree.

For targets that fail evaluation, a *triage agent* creates granular fix tasks in the DAG (e.g., one task per unjustified `sorry`). Both the trace analyzer and the triage agents were motivated by the same observation: delegating targeted analysis to fresh, task-scoped agents produces better results than burdening the long-lived orchestrator, whose response quality degrades over dozens of planning rounds as its context grows—a phenomenon we refer to as *LLM fatigue*.

4.2 Tool design

AUTOFORMBOT equips its agents with tools. Tool servers expose capabilities via the Model Context Protocol (MCP, Anthropic, 2024); the framework converts MCP tool schemas into native function-call definitions for the underlying LLM, so that models interact with tools through their standard function-calling interface. Agents are equipped with a configurable set of MCP tool servers, organized by category:

- **Execution.** A Lean REPL (Morrison, 2024) for running Lean code interactively, and a Lean LSP server (Dressler, 2025) for in-file diagnostics and proof-state queries.
- **Filesystem and search.** Sandboxed filesystem access, grep search, and a MATHLIB search index powered by Loogle (Breitner, 2023) for type-based declaration search.
- **Version control.** Git operations and worktree management (creation, synchronization, cleanup), with shared MATHLIB installation via symlinks to keep worktrees lightweight.
- **Orchestration.** Sub-agent spawning, task dispatch, trackers for tasks and targets, and issues with dependency-aware status lifecycle, scratchpad for persistent notes, job scheduling, and trace inspection for debugging completed runs.
- **Communication.** User-to-agent and agent-to-user communication.
- **Discovery.** Skill loading from accumulated knowledge.

The exact set of tools and permissions granted to an agent depends on its role.

4.3 Coordination Framework

Agents exist within an infrastructure framework that conditions their actions and with which they can interact through their tools.

- **Task tracker.** The persistent DAG of tasks with explicit dependency edges and a fixed status lifecycle (pending $\rightarrow$ in-progress $\rightarrow$ completed, failed or deleted, ready to be dispatched or not).
- **Git worktree isolation.** Every agent operates in a short-lived `git worktree` branching from the shared repository. Completed work is integrated via rebase-then-fast-forward merge, preserving a linear history on the main branch.
- **Concurrent racing.** Several worker agents can race on the same task, each working in its own worktree. The first agent to clear all gates wins.
- **Resource budgeting.** A *resource pool* wraps `asyncio` semaphores with active-count tracking to cap concurrent LLM calls and tool invocations.
- **Process management.** Agents rely on long-lived subprocesses (Lean REPL sessions, inference servers) that require careful lifecycle management. A thread-based session pool provides queue-based load balancing across stateful RPC backends, background monitors enforce per-process memory limits, and a graceful teardown routine cleans up entire process trees on agent completion or failure.
- **Multi-node execution.** The executor abstraction that mediates between the scheduling loop and agent pools has both a local and a distributed implementation.
- **Merge queue.** We use a batched merge queue inspired by [bors-ng contributors \(2017\)](#): pending merges produced by workers are collected, rebased sequentially onto `main`, and verified with a single build over the combined result. If it fails, the queue bisects the batch to isolate the breaking commit, lands the good prefix, and rejects the offender.
- **Escalation protocol.** Workers can file structured escalations that are consumed by the trace analyzer and surfaced to the user (see §4.4).

4.4 Human-in-the-loop and visualization

Though AUTOFORMBOT is designed to operate fully independently, we also provide a convenient visual interface for tracking key metrics (compute spent, number of formalized statements, flagged issues, infrastructure usage, ...), inspecting the graph of logical dependencies, and enabling bidirectional communication with agents through escalation messages and user directives. A screenshot of one of the tabs of the visual interface is shown in Figure 4.

5 Results and Analysis

5.1 ATLAS: verified formal libraries

We have applied AUTOFORMBOT powered by Opus 4.6 to a corpus of **26** open-access mathematical textbooks spanning analysis (real, complex, functional, Fourier, and differential), algebra (abstract and Lie theory), algebraic and differential geometry, algebraic topology, number theory (algebraic, arithmetic, and elliptic curves), combinatorics, partial differential equations, probability and statistics, and theoretical computer science. Human involvement was minimal, ranging from nonexistent to sending a few messages of general advice ("Try decomposing this difficult task into helper lemmas") once a day depending on the book. The pipeline ran for about a week on each book, though increased parallelism can speed up the process.

The resulting library comprises over **45,000** verified Lean 4 declarations and about **500 thousand** lines of code, with the precise decomposition detailed in Table 1. This is comparable in order of magnitude to MATHLIB's 2.1 million lines of code and 308,129 declarations (Li et al., 2026). Each book gives rise to a self-contained Lean project that depends on MATHLIB and builds without errors. Definitions and theorem statements are checked for faithfulness to the source material through the evaluation harness (§3). We release ATLAS as open-source repositories, one per book, with full provenance linking each formal statement back to its source text.

As can be observed in Table 1, none of the books are fully formalized. Most books contain a few particularly

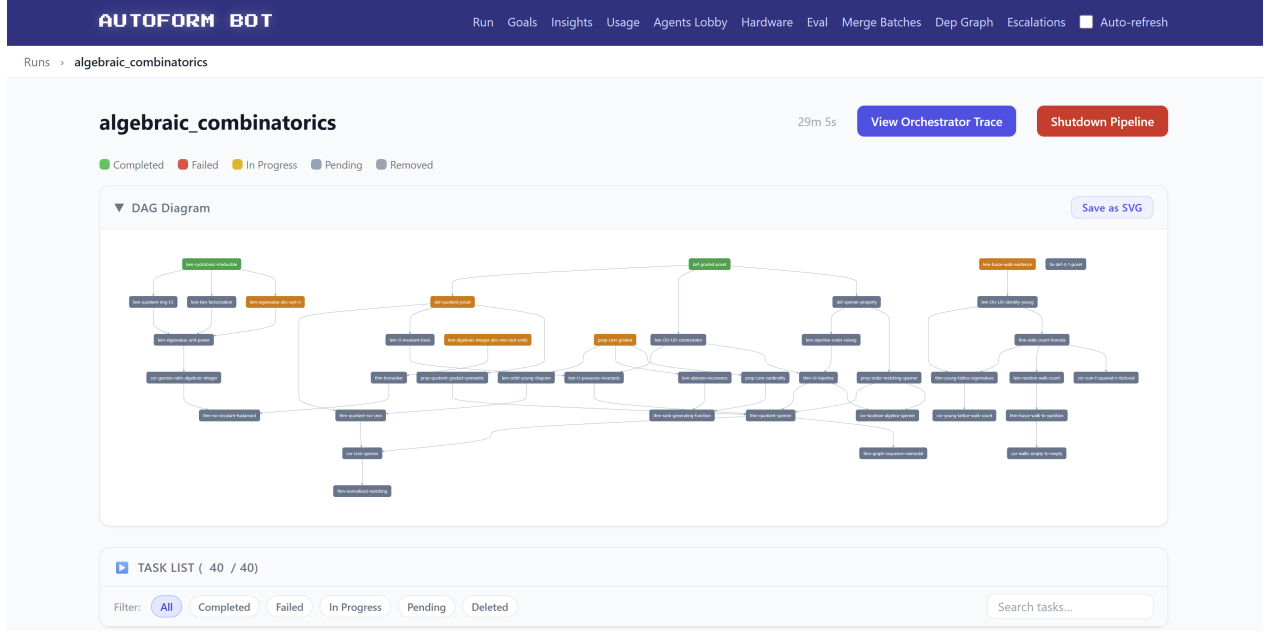


Figure 4 The graph of tasks of a formalization attempt, as shown by AUTOFORMBOT’s visual interface.

challenging statements, often because the required mathematical infrastructure is both absent from MATHLIB and not developed in detail in the book itself; such statements would present similar challenges to human annotators. Our policy was to stop formalization processes once they reached a phase of strongly diminishing returns, where each new success required exponentially more compute, in order to allocate our resources to the coverage of more books. In Figure 5, we report an estimate of the difficulty of each statement in each book, in terms of how much infrastructure required to formalize it is missing from MATHLIB.

Compute costs are dominated by the workers (see Appendix A for a precise decomposition). Though exact numbers depend on the chosen API and the provider’s pricing, we estimate that our pipeline is already cheaper per line of code than expert human annotators, while being faster and more scalable. On the other hand, the overall quality of the output remains inferior to that of expert-written Lean code (as flagged by both our harness and visual inspection by human experts).

Trustworthiness of the evaluation harness Our evaluation harness relies on a combination of mechanical checks and LLMs as judges, which should not a priori be fully trusted. However, direct inspection by human experts (see Appendix G) of the formalization of our example book aligns with the conclusions of our harness, which strengthens our confidence in its judgment.

5.2 Ablations

We evaluate the pipeline along three axes: the underlying language model, the contribution of each feedback component, and the degree of worker parallelism. All experiments target the same book (*Algebraic Combinatorics* by Richard Stanley, 39 targets) since it was among the smallest books and has a moderate difficulty. We report cumulative compute cost in millions of tokens (see also Appendix A).

Model comparison. We run the full pipeline with a single worker per task using either Claude Opus 4.6 or Gemini 3.1 Pro as the backbone (Figure 6). At 1200M tokens, Claude has completed 92% of targets while Gemini reaches only 46%. The gap is visible from the start and, since every other component is identical, is attributable to the model’s ability to code in Lean.

Component ablations. We remove each of the three feedback components independently and compare against the full pipeline (Figure 6). Without the orchestrator loop, a static DAG is created at the start and

Table 1 Per-book formalization results. For each book we report formalized target statements out of the total identified in the source, lines of Lean 4 code (excluding comments and blank lines), and tokens consumed (Appendix A).

Course	Area	Formalized Statements	LoC	Tokens (M)
<i>Algebra Notes I & II</i>	Algebra	151/176 (85.8%)	4,409	1,963
<i>Algebraic Combinatorics</i>	Combinatorics	37/39 (94.9%)	9,343	1,441
<i>Algebraic Geometry I</i>	Alg. Geometry	112/186 (60.2%)	27,393	7,629
<i>Algebraic Topology I</i>	Topology	110/171 (64.3%)	20,143	10,323
<i>An Algorithmist's Toolkit</i>	Combinatorics	131/158 (82.9%)	8,234	2,004
<i>Arithmetic Geometry</i>	Number Theory	266/335 (79.4%)	29,573	11,101
<i>Boolean Functions</i>	Combinatorics	44/108 (40.7%)	7,949	2,327
<i>Buildings</i>	Algebra	44/74 (59.5%)	48,809	20,443
<i>Combinatorial Optimization</i>	Combinatorics	22/36 (61.1%)	7,934	2,476
<i>Complex Variables</i>	Analysis	37/38 (97.4%)	6,225	1,251
<i>Differential Analysis</i>	Analysis	88/113 (77.9%)	23,713	11,743
<i>Differential Geometry</i>	Geometry	112/147 (76.2%)	8,942	1,934
<i>Elliptic Curves</i>	Number Theory	212/360 (58.9%)	22,316	11,058
<i>Fourier Analysis</i>	Analysis	34/38 (89.5%)	6,671	1,186
<i>Geometry of Manifolds</i>	Geometry	40/72 (55.6%)	16,408	6,865
<i>High Dimensional Statistics</i>	Probability & Statistics	65/73 (89.0%)	31,715	975
<i>Intro. to Functional Analysis</i>	Analysis	68/72 (94.4%)	2,006	554
<i>Intro. to PDEs</i>	PDEs	86/105 (81.9%)	20,740	2,972
<i>Lie Groups</i>	Algebra	74/185 (40.0%)	50,594	45,384
<i>Number Theory I</i>	Number Theory	460/576 (79.9%)	54,760	15,424
<i>Probabilistic Methods in Combinatorics</i>	Combinatorics	109/210 (51.9%)	15,604	2,720
<i>Projection Theory</i>	Analysis	73/111 (65.8%)	9,672	2,678
<i>Real Analysis</i>	Analysis	175/177 (98.9%)	2,224	586
<i>Tensor Categories</i>	Algebra	137/229 (59.8%)	29,729	11,338
<i>Theory of Computation</i>	Computer Science	84/118 (71.2%)	10,581	3,580
<i>Theory of Probability</i>	Probability & Statistics	84/100 (84.0%)	8,231	3,201
Total (26 books)		2,855/4,007 (71.3%)	483,918	183,157

execution is guided solely by the trace analyzer and supervisor, losing the ability to re-plan at the level of the whole run. Without the supervisor, the orchestrator receives no structured feedback about target-level quality after merges. Without the trace analyzer, workers lose inter-attempt learning and cannot build on previous failures. All four configurations use three workers per task and Claude Opus 4.6.

Within a matched budget of 600M tokens, the full system reaches 77%. The no-orchestrator variant is the most token-efficient early on, outperforming the full system until ~ 100 M tokens, but plateaus at 64%—indicating the pipeline got stuck without the orchestrator’s ability to re-plan around difficult targets. Removing the supervisor yields 51%: without post-merge quality checks the system keeps hitting a wall with no clear signal on how to fix target-level problems. Removing the trace analyzer reaches 57% but exhausts its compute budget fastest, as workers repeat the same mistakes across attempts.

Parallelism. We vary the number of workers racing on each task—1, 3, or 5—to measure the compute-latency tradeoff (Figure 6). In wall-clock time (dashed lines, top axis), parallelism provides a clear speedup: at 4 hours the 3- and 5-agent configurations reach ~ 62 – 68% while the single-agent configuration is at 44%. Interestingly, 3 and 5 agents also achieve higher scores at lower token budgets, suggesting that parallelism on early, easy tasks reduces wasted serial exploration and lets the pipeline advance faster through the dependency graph.

5.3 Takeaways and insights

Failure modes. We observe several recurring failure patterns:

- **Frontal assault.** Workers repeatedly attempt the same proof strategy despite prior failures, wasting tokens on identical dead ends.
- **Cheating.** Workers find creative ways to circumvent the build system: introducing hidden axioms deep in the codebase, weakening hypotheses, or smuggling `sorry` behind helper lemmas. A particularly disruptive failure mode occurs when workers replace a foundational but complex mathematical object, such as a manifold or a scheme, with an oversimplified formal definition, which can then contaminate many dependent statements. Stricter reviewing curbs this, but workers then hide axioms in increasingly subtle ways, creating an adversarial dynamic that motivates layered review.

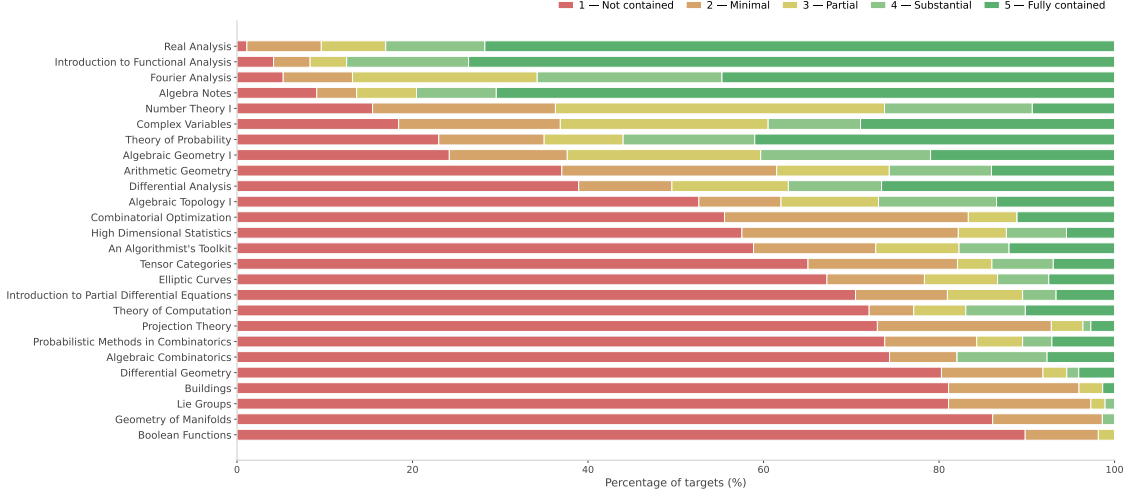


Figure 5 For each statement within each book, we estimate its difficulty in terms of how much infrastructure required to formalize it is missing from MATHLIB, and report these statistics for each book. The rubric used for this estimation is provided in Appendix D.5.

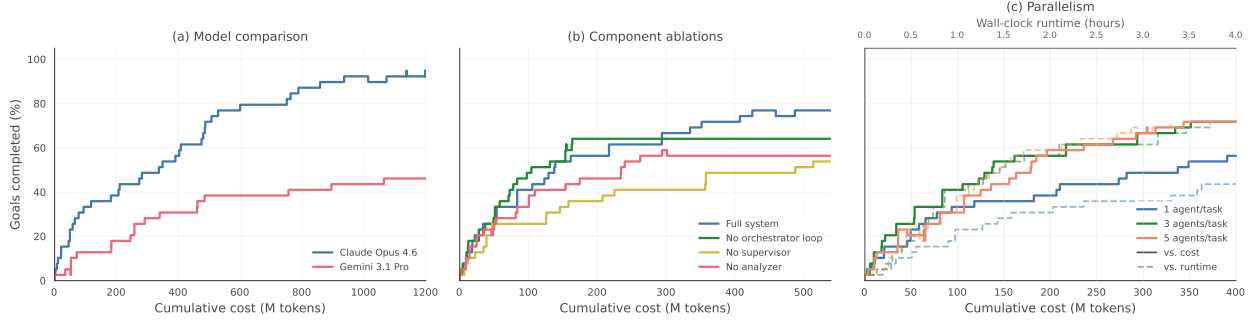


Figure 6 Ablation results on *Algebraic Combinatorics* (39 targets). All plots show goals completed (%) as a function of cumulative token cost. (a) Claude vs. Gemini with the full pipeline. (b) Removing each feedback component independently. (c) Varying workers per task; solid lines show cost, dashed lines show wall-clock runtime (top axis).

- **Infrastructure panic.** When a task requires building substantial infrastructure (e.g., differential forms for manifolds), workers often refuse to proceed, arguing that the task would be too difficult.
- **Orchestrator fatigue.** Over long runs, the orchestrator’s context degrades: it creates coarse-grained tasks with unhelpful prompts, or silently stops dispatching work on difficult targets. Delegating failure analysis to the trace analyzer (Section 5.2) mitigates this.

What worked. Giving the orchestrator progressive-detail tools, a summary view of target completion rates, the ability to drill into individual targets for the latest feedback, and access to task-level error logs, let it make informed re-planning decisions without flooding its context with the full project state at once. Delegating responsibilities to specialized agents rather than burdening a single orchestrator distributes cognitive load and avoids context degradation over long runs. Worker escalations, particularly decomposition requests, are surprisingly informative: they surface false axioms and suggest decomposition strategies that make otherwise intractable tasks solvable.

6 Limitations and Conclusion

We introduced AUTOFORMBOT, a multi-agent system for formalizing mathematical textbooks at scale. By orchestrating hundreds of LLM agents through proven software engineering practices (version control,

review, worktrees, and an issue/task tracker), AUTOFORMBOT produces verified Lean libraries from diverse mathematical sources. We release two artifacts: AUTOFORMBOT (the open-source framework) and ATLAS (verified formal libraries spanning **26** books). Our results suggest that formalizing the core textbook infrastructure of modern mathematics is within reach. This, in turn, opens the door to the automated verification of new mathematical results, and thus to large scale human/machine collaboration.

However, our system relies on frontier LLMs, whose compute costs still remain nontrivial. Moreover, each textbook was formalized *in isolation*, without the careful planning needed to make it maximally compatible with existing MATHLIB infrastructure. Human involvement currently remains necessary to handle such organisational concerns—selecting and ordering books in a dependency-aware manner, bridging mismatched conventions across sources, etc. Our next endeavour will be to bring ATLAS to the level of completeness and standardization required to serve as a proper machine-generated extension of MATHLIB, a process where humans-in-the-loop will likely play a key role.

Broader Impact

Like much research on large language models, this work may have significant social impact. In particular, it contributes to a line of research that could substantially reshape the mathematical profession. Our hope is that such tools will ultimately change the field for the better, freeing mathematicians to focus more on the creative and exploratory aspects of their work.

Acknowledgment

AH is supported by Hi! PARIS and ANR/France 2030 program (ANR-23-IACL-0005). JK and NP thank the Simons Foundation for support through the Collaborative Grant “The Physics of Learning and Neural Computation” as well as support by the NSF through NRT Award 1922658. VC and CA thank Adam Kiezun, Robert Rusch and Shuchao Bi for helpful conversations.

References

- Mohammed Abouzaid, Andrew J. Blumberg, Martin Hairer, Joe Kileel, Tamara G. Kolda, Paul D. Nelson, Daniel Spielman, Nikhil Srivastava, Rachel Ward, Shmuel Weinberger, and Lauren Williams. First Proof, 2026. <https://arxiv.org/abs/2602.05192>.
- Tudor Achim, Alex Best, Alberto Bietti, Mathis Federico, Sergei Gukov, Daniel Halpern-Leistner, et al. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.
- AI-MO. NuminaMath-LEAN. <https://huggingface.co/datasets/AI-MO/NuminaMath-LEAN>, 2025. Hugging Face dataset. 100K competition mathematics problems formalized in Lean 4. Accessed: 2026-04-29.
- Anthropic. Model context protocol, 2024. <https://modelcontextprotocol.io>.
- Anthropic. Claude Opus 4.6 system card. <https://www-cdn.anthropic.com/Odd865075ad3132672ee0ab40b05a53f14cf5288.pdf>, 2026.
- Scott Armstrong and Julia Kempe. Formalization of De Giorgi–Nash–Moser Theory in Lean, 2026. <https://arxiv.org/abs/2604.05984>.
- Scott Armstrong, Julia Kempe, and Remi Munos. Takeaways from the First-Proof trenches. <https://kempejulia1.github.io/1stProof-Attempt/1stProofTakeaways.pdf>, 2026. Accessed: 2026-05-06.
- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- Grzegorz Bancerek, Czesław Byliński, Adam Grabowski, Artur Korniłowicz, Roman Matuszewski, Adam Naumowicz, and Karol Pąk. The role of the Mizar mathematical library for interactive proof development in Mizar. *Journal of Automated Reasoning*, 61(1–4):9–32, 2018. doi: 10.1007/s10817-017-9440-6.

- Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004. ISBN 978-3-540-20854-9. doi: 10.1007/978-3-662-07964-5.
- bors-ng contributors. Bors-ng: A merge bot for GitHub pull requests. <https://github.com/bors-ng/bors-ng>, 2017. GitHub repository.
- Ana Bove, Peter Dybjer, and Ulf Norell. A brief overview of Agda – a functional language with dependent types. In Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel, editors, *Theorem Proving in Higher Order Logics*, volume 5674 of *Lecture Notes in Computer Science*, pages 73–78. Springer, 2009. doi: 10.1007/978-3-642-03359-9_6.
- Joachim Breitner. Loogle: Lean search engine, 2023. <https://loogle.lean-lang.org>.
- Evan Chen, Chris Cummins, Dejan Grubisic, Leopold Haller, Letong Hong, Andranik Kurchinyan, Kenny Lau, Hugh Leather, Seewoo Lee, Aram Markosyan, et al. Fel’s conjecture on syzygies of numerical semigroups. *arXiv preprint arXiv:2602.03716*, 2026.
- Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, et al. Seed-Prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, 2021.
- DeepSeek-AI. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- Oliver Dressler. lean-lsp-mcp: MCP server for the Lean language server, 2025. <https://github.com/oOo0oOo/lean-lsp-mcp>.
- Tony Feng, Trieu H. Trinh, Garrett Bingham, Dawsen Hwang, Yuri Chervonyi, Junehyuk Jung, Joonkyung Lee, Carlo Pagano, Sang-hyun Kim, Federico Pasqualotto, Sergei Gukov, Jonathan N. Lee, Junsu Kim, Kaiying Hou, Golnaz Ghiasi, Yi Tay, YaGuang Li, Chenkai Kuang, Yuan Liu, Hanzhao Lin, Evan Zheran Liu, Nigamaa Nayakanti, Xiaomeng Yang, Heng-Tze Cheng, Demis Hassabis, Koray Kavukcuoglu, Quoc V. Le, and Thang Luong. Towards autonomous mathematics research. *arXiv preprint arXiv:2602.10177*, 2026.
- Fabian Gloeckle, Jannis Limperg, Gabriel Synnaeve, and Amaury Hayat. Abel: Sample efficient online reinforcement learning for neural theorem proving. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS’24*, 2024.
- Fabian Gloeckle, Ahmad Rammal, Charles Arnal, Remi Munos, Vivien Cabannes, Gabriel Synnaeve, and Amaury Hayat. Automatic textbook formalization. *arXiv preprint arXiv:2604.03071*, 2026.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>, 2026.
- Adam Grabowski, Artur Korniłowicz, and Adam Naumowicz. Mizar in a nutshell. *Journal of Formalized Reasoning*, 3(2):153–245, 2010. doi: 10.6092/issn.1972-5787/1980.
- Sidharth Hariharan, Christopher Birkbeck, Seewoo Lee, Ho Kiu Gareth Ma, Bhavik Mehta, Auguste Poiroux, and Maryna Viazovska. A milestone in formalization: The sphere packing problem in dimension 8. *arXiv preprint*, 2026. <https://arxiv.org/abs/2604.23468>.
- John Harrison. HOL Light: An overview. In Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel, editors, *Theorem Proving in Higher Order Logics*, volume 5674 of *Lecture Notes in Computer Science*, pages 60–66. Springer, 2009. doi: 10.1007/978-3-642-03359-9_4.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

- Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi: 10.1038/s41586-025-09833-y.
- Haocheng Ju and Bin Dong. AI for mathematics: Progress, challenges, and prospects. *arXiv preprint arXiv:2601.13209*, 2026.
- Guillaume Lample, Timothee Lacroix, Marie-Anne Lachaux, Aurelien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. Hypertree proof search for neural theorem proving. *Advances in neural information processing systems*, 35:26337–26349, 2022.
- Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The network structure of Mathlib. *arXiv preprint arXiv:2604.24797*, 2026.
- Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A frontier model for open-source automated theorem proving. *arXiv preprint arXiv:2502.07640*, 2025.
- Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, et al. Numina-lean-agent: An open and general agentic reasoning system for formal mathematics. *arXiv preprint arXiv:2601.14027*, 2026.
- Norman D. Megill and David A. Wheeler. *Metamath: A Computer Language for Mathematical Proofs*. Lulu Press, Morrisville, North Carolina, 2019. ISBN 978-0-359-70223-7. <https://us.metamath.org/downloads/metamath.pdf>.
- Scott Morrison. Lean REPL, 2024. <https://github.com/leanprover-community/repl>.
- Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002. ISBN 978-3-540-43376-7. doi: 10.1007/3-540-45949-9.
- OpenAI. GPT-5.4 thinking system card. <https://openai.com/index/gpt-5-4-thinking-system-card/>, 2026.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. *arXiv preprint arXiv:2202.01344*, 2022.
- The Mathlib Community. The Lean mathematical library. *arXiv preprint arXiv:1910.09336*, 2020.
- George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- Josef Urban. 130k lines of formal topology in two weeks: Simple and cheap autoformalization for everyone?, 2026.
- Vasily Vilin. Mathlib conventions. <https://github.com/Vilin97/mathlib-conventions>, 2026. Community-maintained collection of Mathlib coding conventions.
- Haiming Wang, Huajian Xin, Chuanyang Zheng, Li Lin, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jianwei Yin, Zhenguo Li, and Heng Liao. LEGO-Prover: Neural theorem proving with growing libraries. *arXiv preprint arXiv:2310.00656*, 2024.
- Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, et al. Kimina-Prover preview: Towards large formal reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.11354*, 2025.
- Zichen Wang, Wanli Ma, Zhenyu Ming, Gong Zhang, Kun Yuan, and Zaiwen Wen. M2F: Automated formalization of mathematical literature at scale, 2026.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. In *International Conference on Machine Learning (ICML)*, 2024.

- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Huaiyuan Ying, Zijian Shi, Zhengying He, Bohan Gao, Chenyang Chen, Zhicheng Yu, Songrun Song, Qicheng Fan, Yinya Li, Lin Li, et al. Lean workbook: A large-scale Lean problem set formalized from natural language math problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Appendix

A Measure of compute

For a given experiment, we estimate the compute spent by summing over:

- Regular input tokens, i.e. un-cached tokens that were read while producing a response and are not being committed to the cache,
- Cache-read tokens, i.e. cached tokens that were read while producing a response,
- Cache-write tokens, i.e. input tokens being committed to the API’s cache for the first time,
- Output tokens, i.e. tokens produced by the model,

with the following coefficients:

Token type (per 1M)	Multiplier
regular input tokens	1x
cache-read tokens	0.1x
cache-write tokens	1.25x
output tokens	5x

These coefficients match the typical rates applied by frontier model providers through their APIs, and are a reasonable averaged approximation of the true compute spent.

Moreover, we apply a 0.1 multiplicative discount coefficient to the compute spent on the smaller Haiku 4.5 model (mostly used as a document-reading helper), reflecting the difference in size relative to the bigger flagship models.

In the table below, we report the average percentage of total compute used by each agent type.

Category	Ratio (%)
Workers	76.35 $\pm$ 5.71
Reviewers	6.86 $\pm$ 2.38
Supervisor	5.72 $\pm$ 1.54
Orchestrator	4.01 $\pm$ 3.46
Full Eval	3.80 $\pm$ 2.34
Readers	2.00 $\pm$ 0.35
Analyzers	1.28 $\pm$ 1.65

B Per-Book Detailed Results

For each book, we report:

- The name of the course.
- The name of its author.
- The corresponding URL.
- The number of statements (definitions, lemmas, theorems, etc.) in the source material that we are aiming to formalize.
- The number of statements that were successfully formalized (see Subsection 3).
- The number of pages of the source material.
- The number of lines of code of the formalization.
- The number of tokens spent on the formalization attempt (see Appendix A for the exact metric).

Algebra Notes, by Michael Artin & MIT Students

<https://ocw.mit.edu/courses/res-18-011-algebra-i-student-notes-fall-2021/>

Target statements: 176

Formalized statements: 151 (86%)

Pages: 340

Lines of code: 4,409

Tokens (M): 1,963

Algebraic Combinatorics, by Richard Stanley

<https://ocw.mit.edu/courses/18-318-topics-in-algebraic-combinatorics-spring-2006/>

Target statements: 39

Formalized statements: 37 (95%)

Pages: 49

Lines of code: 9,343

Tokens (M): 1,441

Algebraic Geometry I, by Roman Bezrukavnikov

<https://ocw.mit.edu/courses/18-725-algebraic-geometry-fall-2015/>

Target statements: 186

Formalized statements: 112 (60%)

Pages: 63

Lines of code: 27,393

Tokens (M): 7,629

Algebraic Topology I, by Haynes Miller

<https://ocw.mit.edu/courses/18-905-algebraic-topology-i-fall-2016/>

Target statements: 171

Formalized statements: 110 (64%)

Pages: 109

Lines of code: 20,143

Tokens (M): 10,323

An Algorithmists Toolkit, by Jonathan Kelner

<https://ocw.mit.edu/courses/18-409-topics-in-theoretical-computer-science-an-algorithmists-toolkit-fall-2009/>

Target statements: 158

Formalized statements: 131 (83%)

Pages: 171

Lines of code: 8,234

Tokens (M): 2,004

Arithmetic Geometry, by Andrew V. Sutherland

<https://ocw.mit.edu/courses/18-782-introduction-to-arithmetic-geometry-fall-2013/>

Target statements: 335

Formalized statements: 266 (79%)

Pages: 215

Lines of code: 29,573

Tokens (M): 11,101

Boolean Functions, by Dor Minzer

<https://ocw.mit.edu/courses/18-218-topics-in-combinatorics-analysis-of-boolean-functions-spring-2021/>

Target statements: 108

Formalized statements: 44 (41%)

Pages: 86

Lines of code: 7,949

Tokens (M): 2,327

Buildings, by Paul Garrett

<https://www-users.cse.umn.edu/~garrett/m/buildings/book.pdf>

Target statements: 74

Formalized statements: 44 (59%)

Pages: 346

Lines of code: 48,809

Tokens (M): 20,443

Combinatorial Optimization, by Santosh Vempala

<https://ocw.mit.edu/courses/18-433-combinatorial-optimization-fall-2003/>

Target statements: 36

Formalized statements: 22 (61%)

Pages: 66

Lines of code: 7,934

Tokens (M): 2,476

Complex Variables, by Sigurdur Helgason

<https://ocw.mit.edu/courses/18-112-functions-of-a-complex-variable-fall-2008/>

Target statements: 38

Formalized statements: 37 (97%)

Pages: 100

Lines of code: 6,225

Tokens (M): 1,251

Differential Analysis, by Richard Melrose

<https://ocw.mit.edu/courses/18-155-differential-analysis-fall-2004/>

Target statements: 113

Formalized statements: 88 (78%)

Pages: 134

Lines of code: 23,713

Tokens (M): 11,743

Differential Geometry, by Paul Seidel

<https://ocw.mit.edu/courses/18-950-differential-geometry-fall-2008/>

Target statements: 147

Formalized statements: 112 (76%)

Pages: 60

Lines of code: 8,942

Tokens (M): 1,934

Elliptic Curves, by Andrew V. Sutherland

<https://ocw.mit.edu/courses/18-783-elliptic-curves-spring-2021/>

Target statements: 360

Formalized statements: 212 (59%)

Pages: 306

Lines of code: 22,316

Tokens (M): 11,058

Fourier Analysis, by David Jerison

<https://ocw.mit.edu/courses/18-103-fourier-analysis-fall-2013/>

Target statements: 38

Formalized statements: 34 (89%)

Pages: 71

Lines of code: 6,671

Tokens (M): 1,186

Geometry Of Manifolds, by Denis Auroux

<https://ocw.mit.edu/courses/18-966-geometry-of-manifolds-spring-2007/>

Target statements: 72

Formalized statements: 40 (56%)

Pages: 51

Lines of code: 16,408

Tokens (M): 6,865

High Dimensional Statistics, by Philippe Rigollet

<https://ocw.mit.edu/courses/18-s997-high-dimensional-statistics-spring-2015/>

Target statements: 73

Formalized statements: 65 (89%)

Pages: 131

Lines of code: 31,715

Tokens (M): 975

Introduction To Functional Analysis, by Casey Rodriguez

<https://ocw.mit.edu/courses/18-102-introduction-to-functional-analysis-spring-2021/>

Target statements: 72

Formalized statements: 68 (94%)

Pages: 176

Lines of code: 2,006

Tokens (M): 554

Introduction To Partial Differential Equations, by Jared Speck

<https://ocw.mit.edu/courses/18-152-introduction-to-partial-differential-equations-fall-2011/>

Target statements: 105

Formalized statements: 86 (82%)

Pages: 136

Lines of code: 20,740

Tokens (M): 2,972

Representations of Lie Groups, by Pavel Etingof

<https://ocw.mit.edu/courses/18-757-representations-of-lie-groups-fall-2023/>

Target statements: 185

Formalized statements: 74 (40%)

Pages: 161

Lines of code: 50,594

Tokens (M): 45,384

Number Theory I, by Andrew V. Sutherland

<https://ocw.mit.edu/courses/18-785-number-theory-i-fall-2021/>

Target statements: 576

Formalized statements: 460 (80%)

Pages: 318

Lines of code: 54,760

Tokens (M): 15,424

Probabilistic Methods in Combinatorics, by Yufei Zhao

<https://ocw.mit.edu/courses/18-226-probabilistic-methods-in-combinatorics-fall-2022/>

Target statements: 210

Formalized statements: 109 (52%)

Pages: 215

Lines of code: 15,604

Tokens (M): 2,720

Projection Theory, by Lawrence D Guth

<https://ocw.mit.edu/courses/18-156-projection-theory-spring-2025/>

Target statements: 111

Formalized statements: 73 (66%)

Pages: 142

Lines of code: 9,672

Tokens (M): 2,678

Real Analysis, by Casey Rodriguez

<https://ocw.mit.edu/courses/18-100a-real-analysis-fall-2020/>

Target statements: 177

Formalized statements: 175 (99%)

Pages: 92

Lines of code: 2,224

Tokens (M): 586

Tensor Categories, by Pavel Etingof

<https://ocw.mit.edu/courses/18-769-topics-in-lie-theory-tensor-categories-spring-2009/>

Target statements: 229

Formalized statements: 137 (60%)

Pages: 128

Lines of code: 29,729

Tokens (M): 11,338

Theory Of Computation, by Michael Sipser

<https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/>

Target statements: 118

Formalized statements: 84 (71%)

Pages: ~250

Lines of code: 10,581

Tokens (M): 3,580

Theory Of Probability, by Scott Sheffield

<https://ocw.mit.edu/courses/18-175-theory-of-probability-spring-2014/>

Target statements: 100

Formalized statements: 84 (84%)

Pages: ~175

Lines of code: 8,231

Tokens (M): 3,201

C Example Formalizations

We present three representative formalizations from ATLAS, drawn from different books and mathematical domains. For each, we show the original textbook statement, the corresponding Lean 4 formalization, and commentary on the modeling choices and proof strategy.

Example 1: Parseval's equality (Boolean Fourier Analysis)

Source. Claim 2.2 from Minzer, *Topics in Combinatorics: Analysis of Boolean Functions* (MIT 18.218).

Textbook statement. For any $f, g : \{-1, 1\}^n \rightarrow \mathbb{R}$:

1. *Plancherel's equality*: $\langle f, g \rangle = \sum_{S \subseteq [n]} \hat{f}(S) \hat{g}(S)$.
2. *Parseval's equality*: $\|f\|_2^2 = \sum_{S \subseteq [n]} \hat{f}(S)^2$.

```
noncomputable def chi (S : Finset (Fin n))
  (x : Fin n → ZMod 2) : ℝ :=
  (-1 : ℝ) ^ (S.sum fun i => (x i).val)

noncomputable def innerProd
  (f g : (Fin n → ZMod 2) → ℝ) : ℝ :=
  (Finset.univ.sum fun x => f x * g x) / (2 : ℝ) ^ n

noncomputable def fourierCoeff
  (f : (Fin n → ZMod 2) → ℝ)
  (S : Finset (Fin n)) : ℝ :=
  innerProd f (chi S)
```

```

theorem parseval (f : (Fin n → ZMod 2) → ℝ) :
  innerProd f f =
    Finset.univ.sum (fun S => fourierCoeff f S ^ 2) := by
  rw [plancherel]; congr 1; funext S; ring

```

Commentary. The Boolean hypercube $\{-1, 1\}^n$ is modeled as $\text{Fin } n \rightarrow \text{ZMod } 2$, and the Fourier character $\chi_S(x) = \prod_{i \in S} x_i$ becomes $(-1)^{\sum_{i \in S} x_i}$ —mathematically equivalent under the isomorphism $0 \mapsto 1, 1 \mapsto -1$, but better aligned with MATHLIB’s additive group structure on $\text{ZMod } 2$. The proof follows the textbook’s structure: character orthonormality is established via a bit-flipping involution argument (exactly as in the lectures), then Fourier inversion, Plancherel (by bilinearity and orthonormality), and finally Parseval as the special case $g = f$. The file is sorry-free (153 lines for definitions and all supporting lemmas).

Example 2: Mills’ inequality (High-Dimensional Statistics)

Source. Proposition 1.1 from Rigollet, *High-Dimensional Statistics* (MIT 18.657).

Textbook statement. Let $X \sim \mathcal{N}(\mu, \sigma^2)$. For any $t > 0$,

$$\mathbb{P}(X - \mu > t) \leq \frac{1}{\sqrt{2\pi}} \cdot \frac{e^{-t^2/(2\sigma^2)}}{t}.$$

Proof. Reduce to $Z \sim \mathcal{N}(0, 1)$. For $x \geq t > 0$ we have $1 \leq x/t$, so $\int_t^\infty e^{-x^2/2} dx \leq \int_t^\infty \frac{x}{t} e^{-x^2/2} dx = \frac{1}{t} e^{-t^2/2}$.

```

theorem proposition_1_1_mills_inequality
  (t : ℝ) (ht : 0 < t) :
    ∫ x in Ioi t, exp (-(x ^ 2 / 2)) ≤
      t-1 * exp (-(t ^ 2 / 2)) := by
  -- Pointwise: exp(-x^2/2) ≤ (x/t) * exp(-x^2/2)
  have hpointwise : ∀ x ∈ Ioi t,
    exp (-(x ^ 2 / 2)) ≤
      t-1 * (x * exp (-(x ^ 2 / 2))) := ...
  -- Integrate and evaluate RHS via FTC
  calc ∫ x in Ioi t, exp (-(x ^ 2 / 2))
    ≤ ∫ x in Ioi t, t-1 * (x * exp (-(x ^ 2 / 2)))
    := setIntegral_mono_on ...
    _ = t-1 * ∫ x in Ioi t, x * exp (-(x ^ 2 / 2))
    := integral_const_mul _ _
    _ = t-1 * exp (-(t ^ 2 / 2))
    := by rw [integral_Ioi_x_mul_exp t ht]

```

Commentary. The formalization reduces the general $\mathcal{N}(\mu, \sigma^2)$ statement to the core integral bound for the standard Gaussian, exactly as the textbook proof does. The key helper lemma `integral_Ioi_x_mul_exp` computes $\int_t^\infty x e^{-x^2/2} dx = e^{-t^2/2}$ using MATHLIB’s Fundamental Theorem of Calculus, applied to the antiderivative $f(x) = -e^{-x^2/2}$ whose derivative is $f'(x) = x e^{-x^2/2}$. Separate lemmas establish the derivative via `HasDerivAt` and the limit $f(x) \rightarrow 0$ as $x \rightarrow \infty$ via `tendsto_exp_atBot`. The formalization is sorry-free (130 lines).

Example 3: Sperner’s theorem (Algebraic Combinatorics)

Source. Corollary 4.8 from Stanley, *Topics in Algebraic Combinatorics* (MIT 18.312).

Textbook statement. *Definition 4.2.* A graded poset P of rank n has the *Sperner property* if the maximum antichain size equals $\max_i |P_i|$.

Corollary 4.8. The Boolean algebra B_n has the Sperner property. Equivalently, every antichain in $2^{[n]}$ has at most $\binom{n}{\lfloor n/2 \rfloor}$ elements.

```
def HasSpernerProperty (P : GradedPoset α) : Prop :=
  P.maxAntichainCard = P.maxLevelCard

theorem sperner_property_Bn (n : ℕ) :
  (booleanAlgebraGradedPoset n).HasSpernerProperty := ...

theorem sperner_theorem (n : ℕ) :
  ∀ (A : Finset (Finset (Fin n))),
  IsAntichain (· ⊆ ·) (A : Set (Finset (Fin n))) →
  A.card ≤ Nat.choose n (n / 2) := ...
```

Commentary. The Boolean algebra B_n is modeled as `Finset (Fin n)` with the subset ordering, inheriting `MATHLIB`’s `PartialOrder`, `Fintype`, and `GradeMinOrder ℕ` instances. The proof follows the textbook’s linear-algebraic pipeline over three steps: (1) Theorem 4.7 establishes that the *up operator* $U_i f(T) = \sum_{a \in T} f(T \setminus \{a\})$ is injective on level- i functions when $2i < n$ and surjective when $2i \geq n$, using the commutation relation $D_{i+1}U_i - U_{i-1}D_i = (n - 2i)I_i$ and an energy/inner-product argument; (2) Lemma 4.5 converts these into order-matchings via Hall’s marriage theorem, applied to the images of standard basis vectors under the up operator (or its transpose); (3) Proposition 4.4 deduces the Sperner property from the existence of upward and downward matchings by constructing a map from any antichain into the peak level, shown to be injective by case analysis on the relative grades. The concrete form additionally invokes `MATHLIB`’s LYM inequality. The file is sorry-free (1643 lines).

D Evaluation rubrics

Each successfully matched target is scored by three independent LLM judges on a 0–5 integer scale, one judge per rubric. The judges receive the original book statement, the matched Lean declaration with its source code, and tool access to the declaration dependency graph (Section F), which lets them trace sorry chains, inspect structural tags, and investigate suspicious dependencies. Each judge returns a JSON object containing a numeric score and free-text reasoning.

A target passes only if *every* rubric individually meets its threshold. The full scoring criteria for each rubric are given below.

D.1 Faithfulness

Purpose. Evaluates whether the Lean statement faithfully represents the book’s mathematical content.

Threshold. Pass is $\geq 3/5$.

Scoring criteria.

- 5 — Captures the same quantifiers, hypotheses, and conclusion structure as the text; preserves local vs. global scope; domain conditions and set-theoretic structure match the text and are usable.
- 4 — Very close to the text: correct locality, correct hypotheses, correct domain conditions; any extra assumptions do not change the mathematical setting (e.g., adding decidability is fine; changing the function space is not).

- 3** — Core quantifiers, hypotheses, and conclusions match the text; discrepancies are genuinely minor—naming differences, implicit coercions, slightly stronger typeclass assumptions, or equivalent reformulations. The mathematical objects, domain, and all conclusions of the statement are preserved.
- 2** — Some structure matches, but at least one critical mismatch: wrong underlying type or function space (e.g., `BoundedContinuousFunction` vs. C_0), missing a conclusion from a multi-part statement (e.g., missing uniqueness or uniform convergence), adding or dropping hypotheses that change the mathematical content, or formalizing over a different domain than the book specifies.
- 1** — Major deviations: wrong quantifiers, missing key hypotheses, or incorrect conclusion structure; strengthens or weakens the statement substantially.
- 0** — Unrelated to the original statement.

Scoring discipline. The faithfulness prompt includes an explicit anti-inflation guard. If the judge’s own reasoning describes any discrepancy as “meaningful”, “significant”, “non-trivial”, or “notable”, the score must be ≤ 2 , since those words are incompatible with the “genuinely minor” threshold of score 3. Specifically: a wrong underlying type or function space is a domain mismatch (score ≤ 2); a missing conclusion from a multi-part statement is not cosmetic (score ≤ 2). Score 3 is reserved for cases where all mathematical objects, domains, and conclusions are correct, and only superficial details differ.

D.2 Proof integrity

Purpose. Evaluates mathematical correctness of the formalization—whether hypotheses are appropriate and the Lean statement expresses the intended mathematics rather than a lookalike.

Threshold. Pass is $\geq 3/5$.

Scoring criteria.

- 5** — Accurate encoding of all properties; hypotheses no stronger than needed for the informal statement; the Lean statements express the intended mathematics, not a lookalike.
- 4** — Accurate encoding of all core properties; placeholders used only where formalization is genuinely heavy, with correct dependencies; assumptions close to minimal.
- 3** — Key concepts use appropriate Mathlib notions; any gaps are confined to explicitly marked placeholders without changing logical shape; minor definitional mismatches do not undermine intended meaning.
- 2** — At least one central definition or statement is materially wrong or too weak; properties encoded in a way that does not imply the intended property.
- 1** — Key notions encoded incorrectly; statements become false or trivial due to vacuous domains, wrong fields, or meaningless placeholders.
- 0** — Completely wrong or trivially true formalization.

Judge focus. The judge is directed to check whether hypotheses are correct and no stronger than needed, whether the conclusion is correct, whether quantifiers, types, and structures match the intended mathematics, and whether the statement could be vacuously true or trivially satisfied due to wrong domains or fields.

D.3 Code quality

Purpose. Evaluates adherence to Lean and Mathlib coding conventions.

Threshold. Pass is $\geq 3/5$.

Scoring criteria.

- 5 — Follows naming convention mapping textbook items to Lean names (e.g., `theorem_1_17`, `lemma_1_5`, `def_40_3`); comments and docstrings indicate correspondence without polluting Lean syntax; imports minimal and relevant; uses existing Mathlib definitions when available.
- 4 — Clear structure and naming; most declarations correspond cleanly to textbook items; comments and docstrings are informative; imports are largely appropriate; new definitions introduced only when justified.
- 3 — Most items can be traced back to the text; naming mostly consistent; basic organization present but could be clearer; some redundant imports or non-idiomatic redefinitions.
- 2 — Partial mapping to text but inconsistent naming conventions; comments or docstrings missing or unhelpful; imports heavy or redundant; ignores available Mathlib primitives.
- 1 — Naming and structure are chaotic; cannot map Lean items to textbook statements; comments pollute Lean syntax; over-imports or redefines basic Mathlib concepts.
- 0 — Unreadable or structurally broken.

Judge focus. The judge is directed to assess whether each Lean item can be traced back to a textbook item, whether naming follows the project’s conventions, whether comments and docstrings are informative without polluting Lean syntax, whether imports are minimal, and whether the formalization reuses Mathlib definitions when available.

D.4 Score aggregation

A target passes only if every individual rubric meets its threshold ($\geq 3/5$), which ensures a satisfactory degree of completeness.

D.5 Estimation of containment within mathlib

Scoring criteria

- 5 — **Fully contained.** The exact statement (or a direct, trivially equivalent reformulation) exists in mathlib as a named declaration. A user could cite it directly, or with a thin wrapper.
- 4 — **Substantially contained.** The statement is provable in a few lines from existing mathlib lemmas, or a strictly more general version is in mathlib and the target is an immediate specialization.
- 3 — **Partially contained.** Core ingredients (definitions and key supporting lemmas) exist in mathlib, but the headline statement itself is not there and would require nontrivial assembly.
- 2 — **Minimally contained.** Only background definitions or unrelated prerequisites are available; the substantive content is missing.
- 1 — **Not contained.** Mathlib does not contain the statement or the necessary specialized definitions.

E Agent Prompts and Role Descriptions

Each agent in the pipeline is defined declaratively by a system prompt (`prompt.md`) and a configuration file (`config.yaml`) specifying its model, turn budget, tool timeout, and the set of MCP tool servers it may access. All agents use Claude Opus 4.6 unless otherwise noted. Below we summarize the role, tooling, and key instructions for each agent type.

Orchestrator. The orchestrator is a long-lived planning agent (up to 100,000 turns, 400K context window with compaction at 70% utilization) with read-only filesystem access, DAG-store tools, and a persistent TODO list. It never writes Lean code. Its prompt describes the workspace layout (`book/`, `code/`, `skills/`, `reports/`), the full DAG tool API (`list_items`, `add_item`, `update_item`, `delete_item`, `dispatch_task`, `dispatch_ready`), a goal tracker for target-level status, git inspection tools, and an escalation channel.

The prompt specifies a strict task granularity rule: each task covers at most one mathematical statement or one specific fix (a single `sorry`, a single axiom, a single faithfulness issue). It details the first-round workflow (read the book, create one task per formalizable statement with dependency edges from the book’s logical structure, maximize parallelism within each dependency layer) and subsequent-round workflows (read reports from the trace analyzer, update or split failed tasks, cross-reference against goal status). The orchestrator is instructed to prioritize failed goals over pending ones, never silently drop scope, and never retry a failed task with an identical prompt. It also carries the full anti-cheating taxonomy (see below).

Worker. The worker is a short-lived formalization agent (up to 250 turns, 300s tool timeout) with read-write filesystem access, a Lean REPL for interactive type-checking, a Lean LSP server for in-file diagnostics, MATHLIB search tools (Loogle-based), and git. Each worker operates in an isolated git worktree branched from `main`.

The prompt describes the workspace layout and instructs the worker to read skill guides in a specific order before writing any code: first, a task-specific guide (`skills/tasks/{id}/guide.md`) containing lessons from previous failed attempts at the same task; then Lean/MATHLIB API reference files; then workflow best-practice guides. Workers are instructed to use absolute paths, commit with the task ID as prefix, keep helper lemmas public (not `private`), and use `read_and_summarize` for large files to preserve context. The prompt includes an escalation protocol: workers may call `escalate(severity, message)` only for infrastructure failures or tool malfunctions, never for difficult proofs or slow progress.

The prompt includes a detailed “no cheating” section (reproduced in full below), which is shared across the worker, reviewer, and orchestrator prompts.

Reviewer. The reviewer is a code-review agent (up to 40 turns, 120s tool timeout) with a Lean LSP server, read-only filesystem access, and git—but no Lean REPL or MATHLIB search. It evaluates each worker’s changes against five criteria:

1. **Compilation:** run `lean_diagnostic_messages` on changed files.
2. **Task completion:** compare the formalization against the original LaTeX source in `book/`—the reviewer is explicitly instructed to read the book directly and *not* rely on the worker’s docstrings or notes.
3. **Mathematical correctness:** check proof logic and definitions.
4. **Conventions:** proper imports, naming, and structure.
5. **Anti-cheating:** actively check for dishonest proof techniques (see below).

The reviewer is specifically instructed that extra hypotheses not present in the book are “deviations, not justifications”—if the book does not assume a condition, the reviewer must reject unless the worker proves it follows from the book’s assumptions. Responses are structured as either **APPROVED:** `<reason>` or **REJECTED:** `<specific feedback>` with file paths, line numbers, and concrete fix suggestions.

Trace analyzer. The trace analyzer is a persistent agent (up to 100,000 turns) assigned to each failed task, retaining full conversation history across all attempts. It has read-only filesystem access and a trace inspector providing access to execution traces: build errors, reviewer feedback, tool call sequences, agent reasoning, and worker escalations.

The prompt specifies exactly three outputs:

1. A JSON report (`reports/{id}.json`) with task ID, status, attempt count, a 1–2 sentence summary, and up to 3 suggestions.
2. A task-specific skill guide (`skills/tasks/{id}/guide.md`), written only on failure, containing the exact code that almost worked, correct MATHLIB API names, proof strategies to try (and which ones failed), and specific error messages with fixes.
3. An escalation recommendation when the problem is beyond the pipeline’s ability to self-correct.

The prompt includes a hard rule: never suggest `sorry` unless the result has no informal proof in the book.

Merge matcher. The merge matcher (up to 1,000 turns) has read-only filesystem access (restricted to `read_text_file`, `file_grep`, `search_files`, `list_directory`). Given a git diff from a recent merge and a numbered

list of book targets, it inspects the changed files and book source to determine which targets are affected. It returns a JSON response with the list of affected target indices and reasoning.

Matcher (evaluation harness). The matcher agent (up to 1,000 turns) has read-only filesystem access. Given an informal mathematical statement from the textbook, it searches the Lean source directory to find the corresponding declaration. The prompt specifies a search protocol: list the directory structure, use `file_grep` with regex patterns to find declarations, read surrounding context with offset/limit, and check `namespace` blocks to determine fully qualified names. For multi-part statements, the matcher selects the strongest single declaration and notes related ones. Responses include the declaration name, file path, a confidence level (`high`, `medium`, `low`, or `not_found`), and reasoning.

Judge (evaluation harness). The judge agent (up to 1,000 turns) has access to MATHLIB search tools (`mathlib_grep`, `mathlib_find_name`, `mathlib_read_file`) and read-only filesystem access. Given a mathematical statement and its Lean formalization, it scores the formalization on a 0–5 scale according to a specific rubric (faithfulness, proof integrity, or code quality). Each judge invocation receives the rubric criteria, scoring guidelines, and the rubric-specific prompt template as part of its input. The response is a JSON object with a numeric score and reasoning.

Anti-cheating taxonomy. The worker, reviewer, and orchestrator prompts all share a detailed taxonomy of dishonest formalization patterns, reproduced here in full:

- (a) **Trivial statement substitution.** Replacing a theorem’s statement with `True` or another trivially provable proposition while keeping its name and docstring (e.g., `theorem bezout_theorem : True := by trivial`).
- (b) **Encoding theorems as definitions.** Writing `def foo (...) : Prop := <statement>` for something the textbook presents as a theorem. The definition always type-checks (a `Prop` is just a type), so nothing is proved.
- (c) **Smuggling assumptions into structure fields.** Defining a structure whose fields include what should be proved as theorems, then deriving consequences “for free.” Anything stated by the textbook as a “Theorem,” “Proposition,” “Corollary,” or “Lemma” must be a separate Lean theorem proved from the class fields—never a class field itself.
- (d) **Weakening the mathematical content.** Proving a weaker or purely numerical shadow of a theorem instead of the actual result—for instance, proving two vector spaces have the same dimension instead of constructing an isomorphism, or proving a result about integers that encodes a geometric theorem without ever constructing the geometric objects.
- (e) **Modeling avoidance.** Replacing the mathematical objects the textbook works with (e.g., manifolds, schemes, group representations) by simpler algebraic proxies without proving the proxy faithfully represents the real object.
- (f) **Unacknowledged sorry/axiom.** Using `sorry` or `axiom` in helper lemmas that are then called by “proved” theorems, so the top-level theorem appears complete but rests on unproved foundations.

F Structural Tags in the Dependency Graph

As described in Section 3, the evaluation harness builds a declaration-level dependency graph and computes *structural tags* for each declaration by pattern-matching on the proof term. Tags found anywhere in a declaration’s dependency cone propagate upward as *alerts*. Ten tags are currently detected, each flagging a pattern that *may* indicate a formalization defect—though some have legitimate uses:

- **vacuous_body.** The body reduces to `True`, `PUnit.unit`, `trivial`, or a literal. Flags theorems whose statement may have been silently replaced by a tautology.
- **ignores_params.** A lambda abstraction that never references its bound variables. Flags definitions or theorems that accept parameters for type-checking purposes but discard them, often indicating a stub.
- **proof_by_exfalso.** The proof proceeds via `False.elim`, `absurd`, or similar. A valid technique when the context is genuinely contradictory, but can also indicate that an unsatisfiable hypothesis was smuggled into the statement.

- **proof_by_subsingleton.** The proof invokes `Subsingleton.elim`. Legitimate when the target type is provably a subsingleton, but can mask a theorem whose conclusion has been trivialized.
- **returns_assumption.** The body is a single bound variable, or a structure-field projection applied to a bound variable—i.e., the “proof” simply returns one of its hypotheses. Flags theorems where what the textbook states as a result has been encoded as an assumption.
- **field_projection_body.** The body extracts a field from a structure rather than proving a substantive result. Flags declarations where a theorem’s content may have been packed into a structure definition and then projected out.
- **custom_hypothesis_in_type.** The type contains an instance-implicit argument (`[inst : MyClass X]`) for a project-defined class. Flags potential assumption smuggling: the class may bundle axioms that should instead be proved as separate theorems.
- **trivial_constructor.** The body applies a constructor of a project-defined type, but none of the constructor arguments reference project-local declarations. Flags instances assembled from library defaults rather than genuine mathematical content.
- **orphan_class.** A project-defined class with zero instances anywhere in the project. Flags classes likely introduced to bundle assumptions without ever being instantiated by concrete mathematical objects.
- **trivial_instance.** An instance declaration for a project-defined class whose body carries one of the suspicious tags above (e.g. `vacuous_body`, `ignores_params`). Flags instances that satisfy a typeclass vacuously rather than by genuine construction.

The dependency graph serves two roles in the evaluation. First, it enables *inherited-failure filtering*: when a declaration fails the axiom-purity check solely because of `sorryAx` inherited from a transitive dependency that is itself an evaluated target with the same violation, the failure is attributed to the upstream root cause and the downstream declaration is not penalized. This prevents a single `sorry` from cascading into spurious failures across all its dependents. Second, the graph is exposed to the LLM judges as a set of queryable tools—including dependency-health summaries, `sorry-chain` tracing, and suspicious-node listing—allowing them to investigate structural patterns when scoring faithfulness and proof integrity.

G Human Expert Assessment

We asked professional mathematicians with Lean expertise to independently review the machine-generated formalization of *Topics in Algebraic Combinatorics* by R. Stanley, one of our reference textbooks. Below is the full, unedited feedback we received.

Overall verdict. The formalization does not compile in Lean 4.30, but instead targets Lean 4.28. It also uses two explicit axioms—`youngAdjMatrix_eigenvalues_bridge` and `spectral_trace_pow`—which are placeholders for results that are not formalized.

Circulant Hadamard section.

- **Theorem 1** (no circulant Hadamard matrix of order 2^k for $k \geq 3$): Ok.
- **Lemma 2** ($x^{2^{k-1}} + 1$ irreducible over $\mathbb{Q}$): Ok.
- **Unique expression in the ζ -basis**: Ok.
- **Circulant eigenvalue/determinant formula**: Ok.
- **Lemma 3** (Fourier eigenvalues have absolute value $\sqrt{n}$; factorization of 2 in $\mathbb{Z}[\zeta]$): Ok.
- **Lemma 5** ($\mathbb{Z}[\zeta]/(1 - \zeta) \cong \mathbb{F}_2$): Ok.
- **Lemma 6**: Phrasing varies from the book. The formalization gives equation (3) as a hypothesis, when the formula should be proved as a lemma.
- **Corollary 7**: The book’s statement concerns two specific eigenvalues of H ; the formalization states a more general fact, changing the content of the lemma.
- **Lemma 8** (algebraic integer with all conjugates of absolute value one is a root of unity): Ok.
- **Theorem 9 (Kronecker)**: The formalization corrects the book’s statement by adding the hypothesis that α is integral over $\mathbb{Z}$ —this is mathematically necessary, as the original statement is false (e.g. $(3+4i)/5 \in \mathbb{Q}(i)$).

has absolute value one but is not a root of unity). The Lean statement is also slightly more general, allowing the field extension to be generated by multiple roots of unity.

- **Final coefficient comparison (proof of Theorem 1):** Ok; the formalization also fixes a typo in the book.

Section 4: The Sperner property.

- **Definition 4.1:** Ok. Essentially rephrases Mathlib’s built-in `GradeMinOrder`.
- **Definition 4.2** (Sperner property): Ok.
- **Order-matching definition:** Ok.
- **Proposition 4.4** (order-matchings imply rank-unimodal and Sperner): Ok.
- **Lemma 4.5** (injective/surjective order-raising linear operators give matchings): Ok.
- **Boolean up and down operators:** Ok.
- **Lemma 4.6** ($D_{i+1}U_i - U_{i-1}D_i = (n - 2i)I_i$): Ok.
- **Theorem 4.7** (Boolean up operator injectivity/surjectivity): Ok.
- **Corollary 4.8** (B_n is Sperner): Ok.

Section 5: Group actions on Boolean algebras.

- **Definition 5.4** (quotient poset B_n/G): Ok.
- **Proposition 5.6** (B_n/G graded of rank n and rank-symmetric): The Lean file defines “graded” using an axiomatic rank function rather than the book’s definition in terms of maximal chains.
- **Lemma 5.7** (orbit sums form a basis for the fixed subspace): Ok.
- **Lemma 5.8** (Boolean up operator preserves the G -fixed subspace): Ok.
- **Theorem 5.9** (B_n/G is graded, rank-symmetric, rank-unimodal, and Sperner): Ok.
- **Theorem 5.10** (nonisomorphic simple graphs; symmetric and unimodal edge counts): Ok.

Section 6: Young diagrams and q -binomial coefficients.

- **Partition and Young-lattice setup:** Ok; mathematically equivalent, though stated differently.
- **Proposition 6.2** ($L(m, n)$ graded of rank mn , rank-symmetric): The Lean file states rank-symmetry via the existence of an involution, which is equivalent but differs from the book’s definition.
- **Proposition 6.3** ($|L(m, n)| = \binom{m+n}{m}$): The Lean file redefines $L(m, n)$ in a way that simplifies the proof, despite having already defined it differently.
- **q -number, q -factorial, q -binomial definitions:** Done differently than in the book. `qBinomial` is recursive (taking Lemma 6.5 as a definition) rather than a quotient of q -factorials; a separate theorem relates the two.
- **Lemma 6.5** (q -Pascal recurrence): Ok.
- **Theorem 6.6** (rank generating function of $L(m, n)$): Ok.
- **Definition of G_{mn} :** Ok.
- **Lemma 6.8** (every G_{mn} -orbit contains exactly one Young diagram): Ok.
- **Theorem 6.9** ($B_{R_{mn}}/G_{mn} \cong L(m, n)$): The first half is stated as a bijection between $L(m, n)$ and Young diagrams but does not mention the quotient by the group action.
- **Corollary 6.10** ($L(m, n)$ is rank-symmetric, rank-unimodal, and Sperner): Ok.
- **Subset-sum setup:** Ok.
- **Theorem 6.11:** Ok, but slightly generalized to a strictly monotone real sequence instead of an arbitrary positive set.

Section 8: A glimpse of Young tableaux.

- **Young tableaux and f^λ :** Defines `numSYT` by saturated-chain/operator count, not by an explicit type of standard Young tableaux. These are mathematically equivalent, but the tableau object and bijection are not formalized—one of the main nonliteral modelling choices.
- **Hasse-walk count $\alpha(w, \lambda)$ and Equation (40):** Ok.
- **Lemma 8.1:** Ok.
- **Valid λ -word definition:** Ok.
- **Young-lattice up and down operators:** Ok.

- **Lemma 8.2** ($D_{i+1}U_i - U_{i-1}D_i = I_i$): Ok.
- **Equation (43)** ($DU^i = U^iD + iU^{i-1}$): Ok.
- **Theorem 8.3**: Ok.
- **Corollary 8.4** ($\sum_{\lambda \vdash n} (f^\lambda)^2 = n!$): Ok.
- $\beta(\ell, \lambda)$ **setup and facts F1/F2**: Ok.
- **Lemma 8.5** (normal-order expansion coefficients): Ok.
- **Theorem 8.6**: Ok.
- **Corollary 8.7** (closed Hasse walks of length $2m$ from $\emptyset$): Ok.
- **Theorem 8.8** (eigenvalues of the bipartite graph $Y_{j-1,j}$): **Not ok**. The finite-matrix spectral conclusion depends on the explicit axiom `youngAdjMatrix_eigenvalues_bridge`. The Lean predicate `IsEigenvectorYoung` asserts only the eigen-equation without requiring a nonzero vector. The construction works with global functions on all partitions rather than fully level-restricted finite spaces.
- **Corollary 8.9** (alternating walk count): **Not ok**. Lean proves the combinatorial half-trace part, but the spectral trace identity is supplied by the explicit axiom `spectral_trace_pow`, and the eigenvalue input depends on `youngAdjMatrix_eigenvalues_bridge`.

General comments. The main issues are the two added axioms in the last two statements. Overall the project was built in a way that was somewhat inconvenient to use, as the files often imported other files, requiring the project to be set up in a specific way. There was no note that the formalization did not target the newest version of Lean, which should have been included. The Lean formalization fixed some small mistakes in the book and skipped the examples. Overall, most of the formalization is solid, but the hardest statements are not formalized faithfully.